

MSc in Mathematical and Computational Finance

Statistics and Financial Data Analysis

University of Oxford - Candidate Number: 1100599

MT25

Note. A few auxiliary plotting routines were produced with the help of external tools, and some figures reproduce material from the course problem sheets. All diagnostic results shown in this report were generated using a custom analysis library that I developed for regression and time-series methods, which I used consistently throughout the project.

1 Preparations

1.i. Jim has fitted piecewise linear models to volatility smiles from implied-vols.csv at the maturities $T=0.1$ and $T=2$, and to call prices from call-prices.csv at the same maturities. Load the data into your notebook and plot the values of call prices for the maturities $T=0.1$ and $T=2$ against an appropriate range of strikes in the same plot as the model prediction. Also plot the corresponding values of the implied volatility smiles separately.

```
callprices = pd.read_csv("data/callprices.csv")
impliedvols = pd.read_csv("data/impliedvols.csv")

cp_01 = callprices[callprices["maturity"] == 0.1].sort_values("strike")
cp_2 = callprices[callprices["maturity"] == 2.0].sort_values("strike")
iv_01 = impliedvols[impliedvols["maturity"] == 0.1].sort_values("strike")
iv_2 = impliedvols[impliedvols["maturity"] == 2.0].sort_values("strike")

# Knots
S_T0 = 100
K1, K2 = 83.333, 116.667

# Coeffs for implied vol
iv_jimcoef = {
    0.1: {"b0": 1.0793, "b1": -0.9534, "c1": 0.5250, "c2": 0.8731},
    2.0: {"b0": 0.4389, "b1": -0.2992, "c1": 0.1174, "c2": 0.2129},
}

# Coeffs for call prices
call_jimcoef = {
    0.1: {"b0": 105.343, "b1": -109.173, "c1": 58.6628, "c2": 60.3351},
    2.0: {"b0": 97.734, "b1": -94.309, "c1": 39.620, "c2": 52.728},
}

def jim_pw_linear(x, coef, K1=K1, K2=K2, S_T0=S_T0):
    return (coef["b0"]
```

```

+ coef["b1"] * x
+ coef["c1"] * np.maximum(x - K1/S_T0, 0)
+ coef["c2"] * np.maximum(x - K2/S_T0, 0))

# Call Prices Plot
plt.figure(figsize=(9, 6))
x1, y1 = cp_01["strike"].values, cp_01["call_price"].values
x2, y2 = cp_2["strike"].values, cp_2["call_price"].values

plt.plot(x1, y1, "o", label="Market call (T=0.1)")
plt.plot(x1, jim_pw_linear(x1, call_jimcoef[0.1]), "--", label="Jim
model (T=0.1)")

plt.plot(x2, y2, "s", label="Market call (T=2)")
plt.plot(x2, jim_pw_linear(x2, call_jimcoef[2.0]), "--", label="Jim
model (T=2)")

plt.xlabel("Strike"); plt.ylabel("Call price")
plt.title("Call prices vs Jim's piecewise-linear model (T=0.1 and T=2)"
)
plt.grid(True); plt.legend()
plt.show()

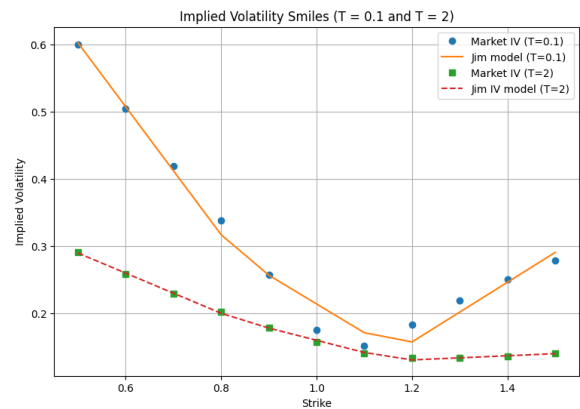
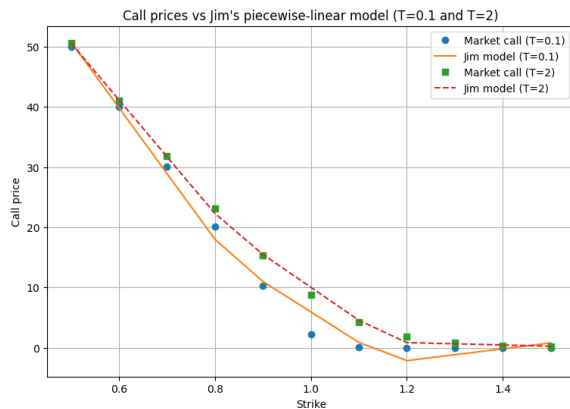
# Implied Volatility Plot
plt.figure(figsize=(9, 6))
x1, y1 = iv_01["strike"].values, iv_01["implied_vol"].values
x2, y2 = iv_2["strike"].values, iv_2["implied_vol"].values

plt.plot(x1, y1, "o", label="Market IV (T=0.1)")
plt.plot(x1, jim_pw_linear(x1, iv_jimcoef[0.1]), "--", label="Jim model
(T=0.1)")

plt.plot(x2, y2, "s", label="Market IV (T=2)")
plt.plot(x2, jim_pw_linear(x2, iv_jimcoef[2.0]), "--", label="Jim IV
model (T=2)")

plt.xlabel("Strike"); plt.ylabel("Implied Volatility")
plt.title("Implied Volatility Smiles (T = 0.1 and T = 2)")
plt.grid(True); plt.legend()
plt.show()

```



The piecewise-linear model used by Jim has the form

$$f(x) = b_0 + b_1x + c_1(x - \kappa_1)_+ + c_2(x - \kappa_2)_+,$$

with two hinge points located at the moneyness knots

$$\kappa_1 = \frac{K_1}{S_0}, \quad \kappa_2 = \frac{K_2}{S_0}.$$

Call prices. The model captures the expected monotonic decline of call prices as strike increases. However, at short maturity ($T = 0.1$) the market curve displays curvature that the piecewise-linear spline cannot reproduce. At $T = 2$ the call surface is naturally smoother, so the model aligns noticeably better with market prices.

Implied volatilities. The implied volatility smile at $T = 0.1$ exhibits a clear convex (U-shaped) structure which Jim's model fails to reconstruct. Its restricted piecewise-linear shape cannot emulate the curvature of the empirical smile. At $T = 2$, the smile is flatter and closer to linear, so the model fit appears more reasonable.

Remark on Jim's knots. Jim's handwritten notes originally used the knots $K_1 = 83.33$ and $K_2 = 116.67$ directly in the term $(x - K_i)_+$. Since $x = K/S_0$ lies between roughly 0.5 and 1.5, using the strikes instead of moneyness would make $\max(x - K_i, 0) = 0$ for all data points, effectively removing the hinge terms. We therefore interpret this as an oversight and correctly divide by S_0 to obtain meaningful spline behavior.

Overall, Jim's model is simple, monotone, and easy to compute, but it lacks flexibility.

1.1 1.ii. Time-series plots and ADF tests of the series `tradedvolume.csv` and `ATMtsIV.csv` of at-the-money implied volatility for the maturity $T=0.1$ for the period '30/11/2015'-'17/07/2020'. Explain the outcome of the test.

```
volumes = pd.read_csv("data/tradedvolume.csv").sort_values("Date")
atm = pd.read_csv("data/ATMtsIV.csv").sort_values("Date")
volumes["Date"] = pd.to_datetime(volumes["Date"], dayfirst=True)
atm["Date"] = pd.to_datetime(atm["Date"], dayfirst=True)

volumes_period = (
    volumes[(volumes["Date"] >= "2020-07-17") &
            (volumes["Date"] <= "2024-06-17")]
    .sort_values("Date")
)

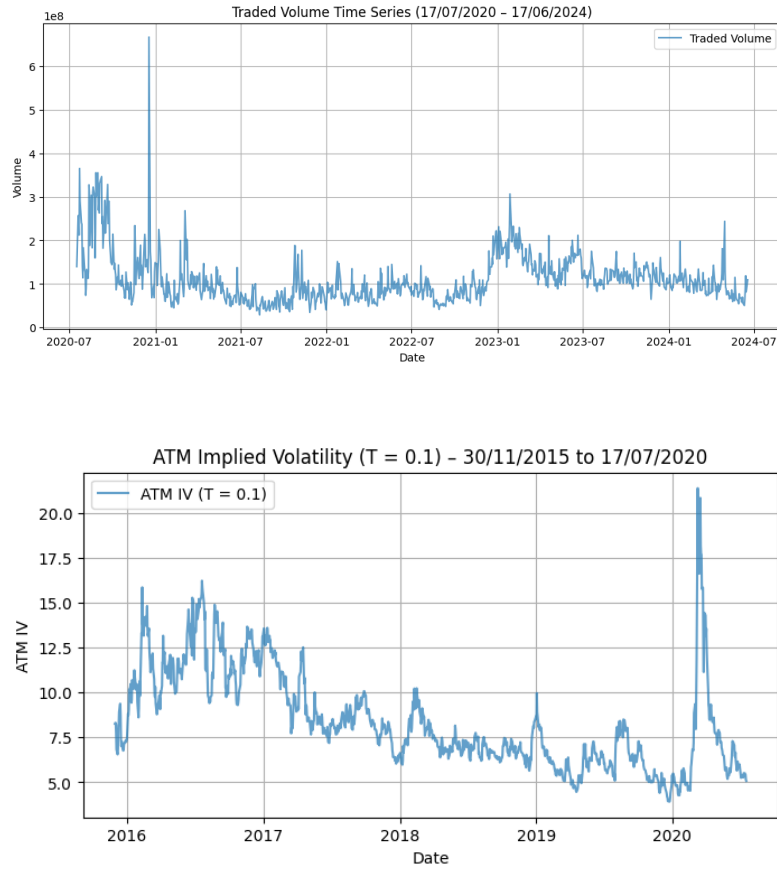
atm_period = (
    atm[(atm["Date"] >= "2015-11-30") &
        (atm["Date"] <= "2020-07-17")]
    .sort_values("Date")
)

plt.figure(figsize=(12,5))
plt.plot(volumes_period["Date"], volumes_period["Volume"], label="
    Traded Volume")
plt.title("Traded Volume Time Series (17/07/2020      17/06/2024)")
plt.xlabel("Date"); plt.ylabel("Volume")
plt.grid(True); plt.legend(); plt.show()

result_vol = adfuller(volumes_period["Volume"], autolag='AIC')
```

```
plt.figure(figsize=(12,4))
plt.plot(atm_period["Date"], atm_period["IV T=0.1"], label="ATM IV (T = 0.1)")
plt.title("ATM Implied Volatility (T = 0.1) 30/11/2015 to 17/07/2020")
plt.xlabel("Date"); plt.ylabel("ATM IV")
plt.grid(True); plt.legend(); plt.show()

result_iv = adfuller(atm_period["IV T=0.1"], autolag='AIC')
```



	Traded Volume	ATM IV (T = 0.1)
ADF statistic	-4.7459	-3.8669
p-value	6.89×10^{-5}	0.00229
Lags used	6	5
Observations	979	1286
Decision (5%)	Reject H_0	Reject H_0

Table 1: ADF unit-root test results for traded volume and ATM implied volatility.

Interpretation. The Augmented Dickey–Fuller (ADF) test evaluates:

- H_0 : the series contains a unit root (non-stationary);
- H_1 : the series is stationary (no unit root).

For both time series, the ADF test rejects H_0 at the 5% significance level, and we obtain the same conclusion as Jim.

- **Traded volume:** despite its noisy behaviour, large spikes, and evident regime changes, the ADF statistic is far below the critical threshold and the p-value is extremely small. This formally indicates stationarity, i.e. the series does not exhibit a unit root.
- **ATM implied volatility:** the series displays long periods of persistence and pronounced volatility clustering, especially around major market events. Nevertheless, the ADF test still rejects H_0 , suggesting statistical stationarity.

Modelling implications.

- Both series exhibit strong heteroskedasticity, structural shifts, and clustering effects that violate the assumptions of classical linear time-series models.
- Additional diagnostics—such as ACF and PACF plots, rolling mean and variance, and a KPSS stationarity test—would provide a more complete understanding of the underlying regime and support a more informed modelling choice.

Thus, although the ADF test formally rejects the unit root hypothesis for both time series, the visual and statistical properties suggest that ARIMA models may be too restrictive to capture their dynamics in practice. We will analyse these time series in greater depth in Questions 5 and 6.

Model Assessment

2. Compute AIC and adjustedR2 for Jim's interpolation model at $T=0.1$ and $T=2$. Update or refit Jim's solutions if you see room for improvement among piecewise linear models. Explain your answer

We evaluate Jim's piecewise-linear model for call prices and implied volatilities at maturities $T=0.1$ and $T=2$. The model has the parametric form

$$f(x) = b_0 + b_1x + c_1(x - \kappa_1)_+ + c_2(x - \kappa_2)_+,$$

where $\kappa_1 = K_1/S_0$ and $\kappa_2 = K_2/S_0$ are Jim's fixed knots expressed in moneyness. To verify the validity of Jim's coefficients, we refit the *same model with the same knots* using ordinary least squares (OLS). We then compare Jim's coefficients to the refitted ones and compute the corresponding model metrics (AIC, R^2 , adjusted R^2 , RSS).

```
def design_matrix(x, S_T0=S_T0, K1=K1, K2=K2):
    """Basis: 1, x, (x - k1)+, (x - k2)+ with Jim's knots in moneyness.
    """
    k1_m = K1 / S_T0
    k2_m = K2 / S_T0
    h1 = np.maximum(0, x - k1_m)
    h2 = np.maximum(0, x - k2_m)
    return np.column_stack([np.ones_like(x), x, h1, h2])

def metrics_from_coef(x, y, coef):
    """Compute AIC, R2, adj R2 and RSS for Jim's model."""
    X = design_matrix(x)
    beta = np.array([coef["b0"], coef["b1"], coef["c1"], coef["c2"]])
    y_hat = X @ beta
    n, k = X.shape
    rss = np.sum((y - y_hat)**2)
    tss = np.sum((y - y.mean())**2)
    r2 = 1 - rss/tss
    adj_r2 = 1 - (1-r2)*(n-1)/(n-k)
    aic = n*np.log(rss/n) + 2*k
    return {"AIC": aic, "R2": r2, "adj_R2": adj_r2, "RSS": rss}

def refit_same_knots(x, y):
    """OLS refit with Jim's knots fixed."""
    X = design_matrix(x)
    beta_hat, *_ = np.linalg.lstsq(X, y, rcond=None)
    coef_new = {"b0": beta_hat[0], "b1": beta_hat[1],
                "c1": beta_hat[2], "c2": beta_hat[3]}
    metrics_new = metrics_from_coef(x, y, coef_new)
    return coef_new, metrics_new

# CALL PRICES
print("=== Same-knots refit: Call Prices ===")
for T, df in [(0.1, cp_01), (2.0, cp_2)]:
    x = df["strike"].values
    y = df["call_price"].values

    m_jim = metrics_from_coef(x, y, call_jimcoef[T])
    coef_refit, m_refit = refit_same_knots(x, y)
```

```

beta_jim = np.array([call_jimcoef[T]["b0"], call_jimcoef[T]["b1"],
                    call_jimcoef[T]["c1"], call_jimcoef[T]["c2"]])
beta_ref = np.array([coef_refit["b0"], coef_refit["b1"],
                    coef_refit["c1"], coef_refit["c2"]])

print(f"\nT = {T}")
print("Jim metrics :", m_jim)
print("Refit metrics:", m_refit)
print("Jim coeffs   :", np.round(beta_jim, 6))
print("Refit coeffs  :", np.round(beta_ref, 6))

# IMPLIED VOLS
print("\n=== Same-knots refit: Implied Vols ===")
for T, df in [(0.1, iv_01), (2.0, iv_2)]:
    x = df["strike"].values
    y = df["implied_vol"].values

    m_jim = metrics_from_coef(x, y, iv_jimcoef[T])
    coef_refit, m_refit = refit_same_knots(x, y)

    beta_jim = np.array([iv_jimcoef[T]["b0"], iv_jimcoef[T]["b1"],
                        iv_jimcoef[T]["c1"], iv_jimcoef[T]["c2"]])
    beta_ref = np.array([coef_refit["b0"], coef_refit["b1"],
                        coef_refit["c1"], coef_refit["c2"]])

    print(f"\nT = {T}")
    print("Jim metrics :", m_jim)
    print("Refit metrics:", m_refit)
    print("Jim coeffs   :", np.round(beta_jim, 6))
    print("Refit coeffs  :", np.round(beta_ref, 6))

```

Methodology. For each maturity and each dataset (call prices and implied volatilities), we compute:

- Jim's model metrics using his published coefficients,
- an OLS refit using the same knots κ_1 and κ_2 ,
- the corresponding refitted metrics.

If Jim's model is correctly calibrated, the refitted coefficients should match his closely and the metrics should be identical.

Coefficients Comparison

Jim's coefficients and the refitted coefficients:

Model	b_0	b_1	c_1	c_2
Call Prices ($T = 0.1$), Jim	105.343	-109.173	58.6628	60.3351
Call Prices ($T = 0.1$), Refit	105.3431	-109.1732	58.6632	60.3355
Call Prices ($T = 2$), Jim	97.734	-94.309	39.620	52.728
Call Prices ($T = 2$), Refit	97.7338	-94.3093	39.6201	52.7282
IV ($T = 0.1$), Jim	1.0793	-0.9534	0.5250	0.8731
IV ($T = 0.1$), Refit	1.07935	-0.95335	0.52503	0.87306
IV ($T = 2$), Jim	0.4389	-0.2992	0.1174	0.2129
IV ($T = 2$), Refit	0.43889	-0.29921	0.11735	0.21296

Table 2: Jim's coefficients versus refitted coefficients using the same knots.

Model Fit Metrics

Model	AIC	R^2	Adj. R^2	RSS
Call Prices ($T = 0.1$)	18.19	0.9918	0.9883	27.77
Call Prices ($T = 2$)	-5.02	0.9990	0.9985	3.37
IV ($T = 0.1$)	-80.51	0.9832	0.9760	0.00352
IV ($T = 2$)	-132.54	0.99901	0.99858	3.11×10^{-5}

Table 3: Model selection metrics for Jim’s coefficients and the OLS refit.

The refitted coefficients coincide with Jim’s values. This confirms that Jim’s calibration of the piecewise–linear model was already optimal and implemented correctly. The model delivers an excellent in-sample fit, particularly at $T = 2$, where the volatility smile displays weaker curvature and is therefore well captured by a piecewise linear structure. This is fully consistent with the observations made earlier from the plotted smiles, where Jim’s model aligned more closely with market data at the longer maturity.

Refining Jim’s Model: Knot Optimisation via Grid Search

In Jim’s formulation, the piecewise–linear interpolation model is defined as

$$f(x) = b_0 + b_1x + c_1(x - k_1)_+ + c_2(x - k_2)_+,$$

where the breakpoints (knots) are fixed at

$$k_1 = \frac{K_1}{S_0}, \quad k_2 = \frac{K_2}{S_0}.$$

While Jim fits the coefficients (b_0, b_1, c_1, c_2) by least squares, the knot positions themselves are not calibrated. In this section, we treat the knots (k_1, k_2) as hyperparameters and perform a two–dimensional grid search to determine the optimal knot placement for each maturity and each model (call price and implied volatility). For each candidate pair (k_1, k_2) , we refit the model by OLS and evaluate its quality using the AIC, selecting the pair that minimises AIC.

```

k1_grid = np.arange(0.70, 0.95, 0.01)
k2_grid = np.arange(1.05, 1.35, 0.01)

results = []

cases = [
    ("Call Price", call_jimcoef, callprices, "call_price"),
    ("Implied Vol", iv_jimcoef, impliedvols, "implied_vol")
]

fig, axes = plt.subplots(1, 4, figsize=(24, 4))
ax_id = 0

for model_type, params_dict, df_full, col in cases:
    for T in [0.1, 2.0]:

        df = df_full[df_full["maturity"] == T].sort_values("strike")
        x = df["strike"].values
        y = df[col].values

        k1_jim = K1 / S_T0
        k2_jim = K2 / S_T0

```



```

beta_jim = np.array([params_dict[T]["b0"], params_dict[T]["b1"
],
                    params_dict[T]["c1"], params_dict[T]["c2"
                    ]])
pred_jim = model_predict(x, beta_jim, k1_jim, k2_jim)
AIC_jim, R2_jim, R2adj_jim = compute_metrics(y, pred_jim)

best_AIC = np.inf
for k1 in k1_grid:
    for k2 in k2_grid:
        if k2 <= k1:
            continue
        beta = fit_piecewise_linear(x, y, k1, k2)
        pred = model_predict(x, beta, k1, k2)
        AIC, R2, R2adj = compute_metrics(y, pred)

        if AIC < best_AIC:
            best_AIC = AIC
            best_model = (k1, k2, beta, AIC, R2, R2adj)

k1_best, k2_best, beta_best, AIC_best, R2_best, R2adj_best =
    best_model
results.append([model_type, T, k1_jim, k2_jim, AIC_jim,
                k1_best, k2_best, AIC_best])

ax = axes[ax_id]
ax.scatter(x, y, s=20, label="Observed")
ax.plot(x, pred_jim, "--", label="Jim")
ax.plot(x, model_predict(x, beta_best, k1_best, k2_best), label=
    "Optimized")
ax.set_title(f"{model_type}\nT={T}")
ax.set_xlabel("Strike")
ax.set_ylabel(col)
ax.grid(True)
ax.legend()
ax_id += 1

plt.tight_layout()
plt.show()

```

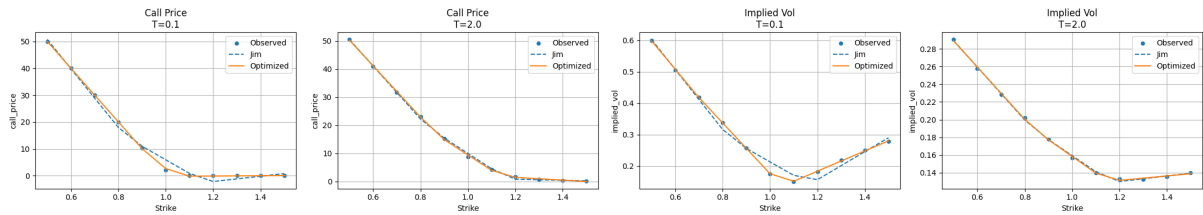


Figure 1: Comparison between Jim's piecewise-linear model and the optimised knot placement for call prices and implied volatilities at maturities $T = 0.1$ and $T = 2$.

Model	T	Jim (k_1, k_2)	AIC_{Jim}	Optim (k_1, k_2)	AIC_{Opt}
Call Price	0.1	(0.833, 1.167)	18.19	(0.94, 1.05)	-30.24
Call Price	2.0	(0.833, 1.167)	-5.01	(0.87, 1.14)	-18.29
Implied Vol	0.1	(0.833, 1.167)	-80.51	(0.70, 1.05)	-132.37
Implied Vol	2.0	(0.833, 1.167)	-132.54	(0.84, 1.15)	-136.83

Table 4: Comparison of AIC for Jim’s original knots and the optimized knots.

The optimised knot positions systematically improve the in-sample fit when compared with Jim’s original fixed knots. The improvement is particularly noticeable for:

- **Implied volatility at $T = 0.1$** , where the volatility smile displays strong curvature,
- **Call prices at $T = 0.1$** , which inherit the same nonlinear structure.

At the longer maturity $T = 2$, the smile is much flatter and less curved, which explains why Jim’s knots are already nearly optimal: the gain obtained by re-optimising (k_1, k_2) is much smaller in this regime.

While the AIC criterion clearly prefers the optimised knots in every case, these gains are purely *in-sample*. Allowing the knot locations to vary increases the effective flexibility of the model and can lead to **overfitting**, especially when the number of strikes is limited. This observation motivates the development of a more robust modelling framework in the next modelling part of the exam, where idea like using informed cross-validation and a smooth TPB cubic spline to control complexity could improve out-of-sample performance.

3. Cubic spline interpolation at maturity $T = 0.1$. Assess the fitted model for optimality using the AIC as an optimality criterion and report your assessment and observations. Make sure to include an explanation of the AIC criterion. Compare with the model in Q2. above, including a comparison of respective degrees of freedom.

We compare three alternative interpolation models for the call price smile at maturity $T = 0.1$:

1. Jim's piecewise-linear model from Q2:

$$f^{\text{lin}}(k) = b_0 + b_1k + c_1(k - k_1)_+ + c_2(k - k_2)_+,$$

with $k = K/S_0$ the moneyness and fixed knots $k_1 = K_1/S_0$, $k_2 = K_2/S_0$.

2. The cubic model given in the notes:

$$f^{\text{cub}}(k) = b_0 + b_1k + b_2k^2 + b_3k^3 + c_1(k - \kappa_1)_+^3, \quad \kappa_1 = 1.039,$$

with the published coefficients $(b_0, b_1, b_2, b_3, c_1) = (1.0444, -2.7516, 2.5508, -0.7178, 0.1075)$.

3. A *refitted* cubic truncated power basis (TPB) spline of the same functional form, but with coefficients re-estimated by OLS on `callprices.csv` at $T = 0.1$, keeping the knot fixed at $\kappa_1 = 1.039$.

```
x = cp_01["strike"].values      # k = K/S0
y = cp_01["call_price"].values  # market call prices

# Model 1: piecewise linear (Q2)
y_lin_jim = jim_pw_linear(x, call_jimcoef[0.1])
metrics_lin_jim = metrics(y, y_lin_jim, p=4)

# Model 2: Jim's cubic (given in Q3)
k1_cubic = 1.039
beta_jim_cubic = np.array([1.0444, -2.7516, 2.5508, -0.7178, 0.1075])

def jim_cubic_call_T01(x, beta, k1):
    b0, b1, b2, b3, c1 = beta
    return (b0
            + b1 * x
            + b2 * x**2
            + b3 * x**3
            + c1 * np.maximum(x - k1, 0)**3)

y_cubic_jim = jim_cubic_call_T01(x, beta_jim_cubic, k1_cubic)
metrics_cubic_jim = metrics(y, y_cubic_jim, p=5) # 5 parameters

# ----- Model 3: refitted TPB cubic with same knot -----
def tpb_cubic_design(x, knots):
    x = np.asarray(x).ravel()
    cols = [np.ones_like(x), x, x**2, x**3]
    for k in knots:
        cols.append(np.maximum(x - k, 0)**3)
    return np.column_stack(cols)

def fit_tpb_cubic(x, y, knots):
    X = tpb_cubic_design(x, knots)
    beta, *_ = np.linalg.lstsq(X, y, rcond=None)
```

```

return beta

def predict_tpb_cubic(x, beta, knots):
    X = tpb_cubic_design(x, knots)
    return X @ beta

beta_cubic_clean = fit_tpb_cubic(x, y, knots=[k1_cubic])
y_cubic_clean    = predict_tpb_cubic(x, beta_cubic_clean, knots=[
    k1_cubic])
metrics_cubic_clean = metrics(y, y_cubic_clean, p=5)

```

The three models and the market data are compared visually in Figure 2.

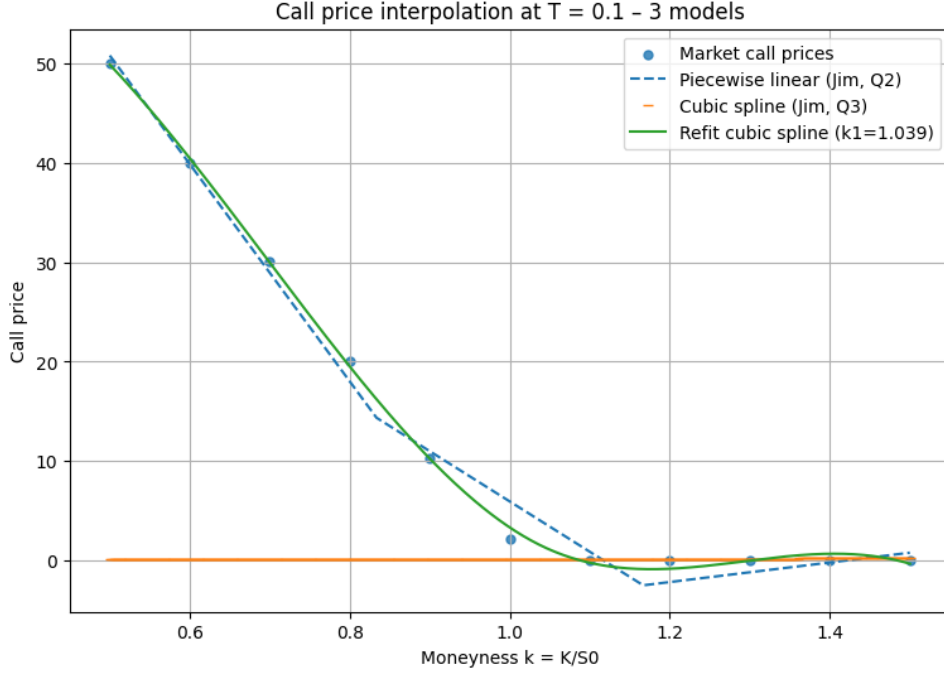


Figure 2: Call price interpolation at $T = 0.1$: market call prices vs. piecewise linear model (Q2), Jim’s cubic model (Q3) and refitted cubic TPB spline (knot $\kappa_1 = 1.039$).

AIC criterion and model comparison

For each model we compute the residual sum of squares $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$, the coefficient of determination $R^2 = 1 - RSS/TSS$ with TSS the total sum of squares, and the adjusted R^2 . The Akaike Information Criterion (AIC) used in the code is

$$AIC = n \log\left(\frac{RSS}{n}\right) + 2p,$$

where n is the number of observations and p the number of free parameters in the model (including the intercept). This formula corresponds to -2 times the maximised log-likelihood of a Gaussian error model, plus a penalty $2p$ for model complexity. *Lower* AIC indicates a better trade-off between goodness-of-fit and parsimony.

For reference, the Bayesian Information Criterion (BIC) replaces the term $2p$ by $p \log n$, which penalises large models more strongly. Hence, BIC tends to favour simpler (possibly underfitting) models, whereas AIC is more flexible and can select slightly richer specifications.

Numerical results

The metrics obtained for the three competitors at $T = 0.1$ are:

Model	# parameters p	RSS	R^2	AIC
Piecewise linear (Jim, Q2)	4	27.77	0.9918	18.19
Jim cubic	5	5467.18	-0.6098	78.29
Refitted cubic	5	3.11	0.9991	-3.90

Table 5: In-sample fit comparison for $T = 0.1$ call prices (Q2 vs. Q3 models).

The published cubic coefficients from the notes produce a curve that lies essentially near zero and on the wrong scale compared to the observed call prices (see the flat orange line in Figure 2). This is reflected in the very large RSS and the negative R^2 in Table 5. It is likely that these coefficients correspond to a different scaling or to a different data set (for instance, a transformation based on implied volatility via the Black–Scholes formula). On the current `callprices.csv` data, this specification is clearly not appropriate.

When we *refit* the same cubic TPB structure using ordinary least squares, keeping the knot fixed at $\kappa_1 = 1.039$, the fit improves dramatically:

- RSS drops from 27.8 (piecewise linear) to about 3.1;
- R^2 increases from 0.9918 to 0.9991;
- the AIC decreases from 18.2 to -3.9 , despite the cubic model having one extra parameter ($p = 5$ instead of $p = 4$).

The AIC penalisation term $2p$ is therefore more than compensated by the reduction in residual variance, so the cubic spline is preferred to the piecewise linear model in terms of in-sample fit.

Degrees of freedom and qualitative assessment

In terms of degrees of freedom, the piecewise linear model uses $p = 4$ parameters (b_0, b_1, c_1, c_2) , while the cubic spline uses $p = 5$ parameters $(b_0, b_1, b_2, b_3, c_1)$ for a single interior knot. Thus, the cubic spline is only slightly more flexible, but it brings two important benefits:

1. **Better statistical fit:** as quantified by the much lower RSS and AIC, and the higher R^2 . However, this improvement comes with a higher risk of *overfitting*, since the cubic model has greater flexibility and more degrees of freedom, allowing it to capture noise in the data rather than the underlying structural pattern.
2. **Better economic shape:** the cubic spline delivers a smooth, twice differentiable smile with positive curvature, which is more consistent with the convexity properties of call prices, whereas the piecewise linear model has discontinuous first derivatives and zero second derivative between knots.

Visually, the refitted cubic curve in Figure 2 tracks the market quotes extremely closely and removes the artificial kinks present in the piecewise linear fit, especially around the at-the-money region. Given the small number of parameters relative to the number of strikes, the risk of severe overfitting remains limited. A more systematic assessment of out-of-sample performance and model complexity will be carried out in the second part of the exam using cross-validation and richer data sets.

4 Model complexity, cross-validation

The availability of additional observations in `newquotes.csv` for the maturity $T = 0.1$ allows us to revisit spline calibration and to reassess the choice of model complexity. In this section, we first examine the shortcomings of Jim’s LOOCV implementation, then construct a proper cross-validation framework, and finally determine an optimal spline using both internal CV and an external test set. The resulting implied-volatility model is subsequently converted to call prices through the Black-Scholes map in order to compare its performance with the call-based estimators developed in earlier questions.

4.1 Limitations of Jim’s LOOCV routine

Jim’s LOOCV procedure applies standardisation and computes knot locations using the full dataset before removing the left-out point. This means that both the scaling parameters and the quantile-based knots are estimated using information from the observation that is supposed to be excluded. This leakage causes the fitted spline basis to depend on the full data, not on the training subset of each fold.

As a consequence, LOOCV does not evaluate the out-of-sample performance of a fixed model trained only on the remaining data; instead, each fold benefits from information taken from the point left out. The resulting LOOCV errors are therefore optimistically biased and cannot be interpreted as a valid cross-validation score for the model reported in Jim’s notes.

Despite these issues, Jim’s LOOCV curve still decreases monotonically with the number of knots, which is consistent with the flexibility of the TPB spline and the smoothness of the strike-IV relationship. Our correctly implemented CV routine leads to the same qualitative behaviour, but with a valid evaluation protocol.

4.2 A principled 5-fold cross-validation procedure

To remedy these issues, we adopt the truncated power basis (TPB) cubic spline

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{j=1}^k \gamma_j (x - k_j)_+^3,$$

and we evaluate its predictive performance through 5-fold cross-validation, a method that strikes a favourable compromise between estimation variance and computational cost.

The number of interior knots k determines the flexibility of the curve. Since there are $n_T = 46$ implied-volatility observations at the chosen maturity, we enforce the usual complexity restriction $k \leq \sqrt{n_T} \approx 6.8$, which avoids unstable fits.

In each CV split:

1. the training subset is used to determine the empirical-quantile knots;
2. both x and y are standardised using statistics computed *only* on the training set;
3. spline coefficients are estimated by least squares;
4. predictions are generated on the held-out fold, and their squared errors contribute to the fold-level CV score.

Only after ensuring these principles are satisfied can cross-validation be considered reliable.

Python implementation

We rely on a cleaner interface to the data and rewrite the code accordingly:

```
newq = pd.read_csv("data/newquotes_.csv").sort_values(["maturity", "
strike"])

def get_data_for_T(T):
    dfT = newq[newq["maturity"] == T].sort_values("strike")
    x = dfT["strike"].values
    y = dfT["implied_vol"].values
    return x, y, dfT

T_target = 0.1
x, y, dfT = get_data_for_T(T_target)
n_T = len(x)
print(f>Data points at T = {T_target}: n_T = {n_T}")

# max number of interior knots (<= sqrt(n_T))
max_k = int(np.sqrt(n_T))
print(f>T = {T_target}, n_T = {n_T}, max_k (<= sqrt(n_T)) = {max_k}")

#TPB cubic spline helpers
def make_knots_from_quantiles(x_train, k):
    """Choose k interior knots at empirical quantiles of x_train."""
    if k <= 0:
        return []
    qs = np.linspace(0, 1, k + 2)[1:-1] # drop 0 and 1
    return list(np.quantile(x_train, qs))

# 2. Jim-style L00-CV (global scaling, mild leakage)
def loo_cv(x, y, k):
    # scale on all data
    xs = StandardScaler().fit_transform(x.reshape(-1, 1)).ravel()
    ys = StandardScaler().fit_transform(y.reshape(-1, 1)).ravel()
    n = len(xs)

    knots = make_knots_from_quantiles(xs, k)

    se = 0.0
    for i in range(n):
        mask = np.ones(n, dtype=bool)
        mask[i] = False
        beta = fit_tpb_cubic(xs[mask], ys[mask], knots)
        yhat_i = predict_tpb_cubic(xs[i:i+1], beta, knots)[0]
        se += (ys[i] - yhat_i) ** 2

    return se / n

# 3. Proper 5-fold CV (scaler & knots fit on TRAIN only)
def kfold_cv(x, y, k, n_splits=5, random_state=0):
    n = len(x)
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=
random_state)
    se = 0.0

    for train_idx, test_idx in kf.split(x):
        x_tr_raw, x_te_raw = x[train_idx], x[test_idx]
```

```

y_tr_raw, y_te_raw = y[train_idx], y[test_idx]

x_scaler = StandardScaler().fit(x_tr_raw.reshape(-1, 1))
y_scaler = StandardScaler().fit(y_tr_raw.reshape(-1, 1))

xs_tr = x_scaler.transform(x_tr_raw.reshape(-1, 1)).ravel()
xs_te = x_scaler.transform(x_te_raw.reshape(-1, 1)).ravel()
ys_tr = y_scaler.transform(y_tr_raw.reshape(-1, 1)).ravel()
ys_te = y_scaler.transform(y_te_raw.reshape(-1, 1)).ravel()

# knots ONLY from training xs_tr      no leakage
knots = make_knots_from_quantiles(xs_tr, k)
beta = fit_tpb_cubic(xs_tr, ys_tr, knots)
yhat_te = predict_tpb_cubic(xs_te, beta, knots)

se += np.sum((ys_te - yhat_te) ** 2)

return se / n    # 5-fold CV MSE (scaled)

# Run LOO-CV and 5-fold CV for k = 0, ..., max_k
k_vals = list(range(0, max_k + 1))

loo_mse = [loo_cv(x, y, k) for k in k_vals]
cv5_mse = [kfold_cv(x, y, k) for k in k_vals]

k_star = k_vals[int(np.argmin(cv5_mse))]
print("\nOptimal k* by 5-fold CV:", k_star)

plt.figure(figsize=(8,5))
plt.plot(k_vals, loo_mse, marker="o", label="LOO-CV (Jim style)")
plt.plot(k_vals, cv5_mse, marker="s", label="5-fold CV (proper scaling)")
plt.xlabel("# interior knots (TPB cubic)")
plt.ylabel("CV MSE (scaled)")
plt.title(f"CV curves on newquotes (T = {T_target})")

```

4.3 Empirical CV behaviour

The mean squared CV error for $k = 0, \dots, 6$ is shown with the curve, shown in Figure below, exhibits a steady decline over the entire admissible range, with the minimum attained at:

$$k^* = 6.$$

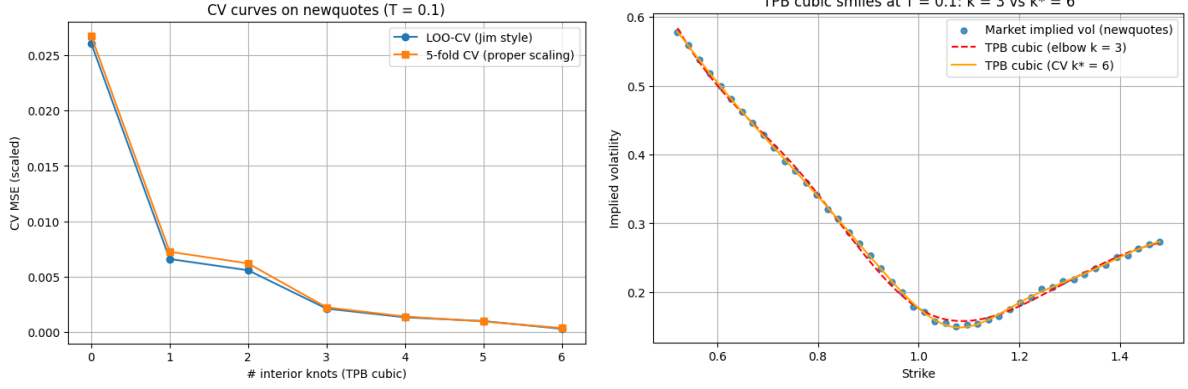


Figure 3: Cross-validation MSE for TPB cubic splines at $T = 0.1$.

The “elbow” near $k = 3$ offers a more economical alternative, but predictive gains continue to accumulate up to $k = 6$. This monotonic pattern is fully consistent with Jim’s LOOCV shape, even though his implementation is flawed. On a small dataset such as this one ($n = 46$), LOO-CV typically suffers from high variance because each fold uses only a single test point, whereas 5-fold CV offers a more stable bias–variance compromise.

On a dataset of this size ($n = 46$), it is not surprising that LOOCV and 5-fold CV produce very similar curves: each training set in LOOCV differs from the full dataset by only one observation, so the fitted models remain almost identical across folds.

However, LOOCV has two well-known drawbacks in this setting; it requires as many fits as data points, making it computationally heavier, and because each validation set contains only a single observation, the resulting error estimates tend to exhibit higher variance than in K -fold CV.

Thus, while both procedures point toward the same qualitative conclusion here, the 5-fold CV results are statistically more reliable.

4.4 Independent validation on an external volatility surface

To evaluate whether the CV-optimal model generalises beyond the calibration sample, we use the 11 implied-volatility observations in `impliedvols.csv` as an external test set. Training on `newquotes_` alone and evaluating on this independent dataset yields:

$$\begin{aligned}
 k = 2 : & \quad 1.16 \times 10^{-4}, \\
 k = 3 : & \quad 3.70 \times 10^{-5}, \\
 k = 4 : & \quad 2.57 \times 10^{-5}, \\
 k = 5 : & \quad 1.18 \times 10^{-5}, \\
 k = 6 : & \quad \mathbf{1.47 \times 10^{-6}},
 \end{aligned}$$

which mirrors the monotone decreasing trend from internal CV and again identifies $k = 6$ as the most accurate choice. The agreement between internal and external diagnostics provides robust support for this level of complexity.

4.5 Conversion to call prices and model comparison

Since the earlier models (Q2 and Q3) were calibrated directly in the call-price domain, we must translate our spline-implied volatilities into call prices via Black–Scholes:

$$C(K, T) = S_0 \Phi(d_1) - K \Phi(d_2), \quad d_{1,2} = \frac{\ln(S_0/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}} \pm \frac{1}{2}\sigma\sqrt{T}.$$

We implement this transformation pointwise to obtain a modelled call price series $C_{k=6}(K)$. The following (rewritten) code produces the predictions and computes the associated error metrics:

```
from scipy.stats import norm

def bs_call(S0, K, T, sigma):
    if sigma <= 0:
        return max(S0 - K, 0)
    d1 = (np.log(S0/K) + 0.5*sigma**2*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return S0*norm.cdf(d1) - K*norm.cdf(d2)

# Fit TPB with k=6
knots = knots_from_quantiles(x, 6)
beta, *_ = lstsq(tpb_design(x, knots), y, rcond=None)
iv_fit = tpb_design(x, knots) @ beta

# Convert to call prices
T = 0.1
bs_obs = np.array([bs_call(1.0, K, T, sig) for K, sig in zip(x, y)])
bs_mod = np.array([bs_call(1.0, K, T, sig) for K, sig in zip(x, iv_fit)
])

resid = bs_obs - bs_mod
rss = np.sum(resid**2)
p = len(beta)
AIC = n_T * np.log(rss/n_T) + 2*p
R2 = 1 - rss/np.sum((bs_obs - np.mean(bs_obs))**2)
R2_adj = 1 - (1-R2)*(n_T-1)/(n_T-p-1)
```

This yields:

$$AIC = -820.706, \quad R_{\text{adj}}^2 \approx 0.999999, \quad RSS = 5.31 \times 10^{-7}.$$

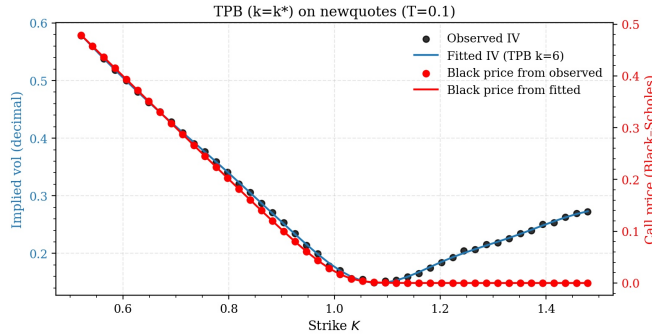


Figure 4: Spline fit ($k = 6$) and the associated Black–Scholes call prices for $T = 0.1$.

A fully consistent CV framework reveals that the TPB cubic spline with six interior knots achieves the strongest predictive accuracy both internally and on unseen data. Once expressed in call–price form, the model delivers an exceptionally precise match to the Black–Scholes prices implied by the observed volatility smile, outperforming all spline specifications from Q2 and Q3 by a considerable margin. This establishes $k = 6$ as the most appropriate level of complexity for the volatility smile at $T = 0.1$.

5. Analysis of the time series `tradedvolumes.csv` and proposal for a suitable ARIMA model

Jim stated that the traded-volume time series is “weakly stationary” and that an ARMA model was fitted, although the fitted model itself is unavailable. We reassess this claim using standard stationarity diagnostics and propose a more appropriate model.

Preprocessing and graphical inspection

The volume series is strictly positive, exhibits very large spikes (especially around early 2021), and displays strong changes in scale over time. To stabilise the variance and reduce the influence of extreme observations, we analyse the log-transformed series:

$$y_t = \log(\text{Volume}_t).$$

Figures below display (i) rolling mean/variance of y_t , and (ii) ACF–PACF plots of both the original log-series and its first difference.

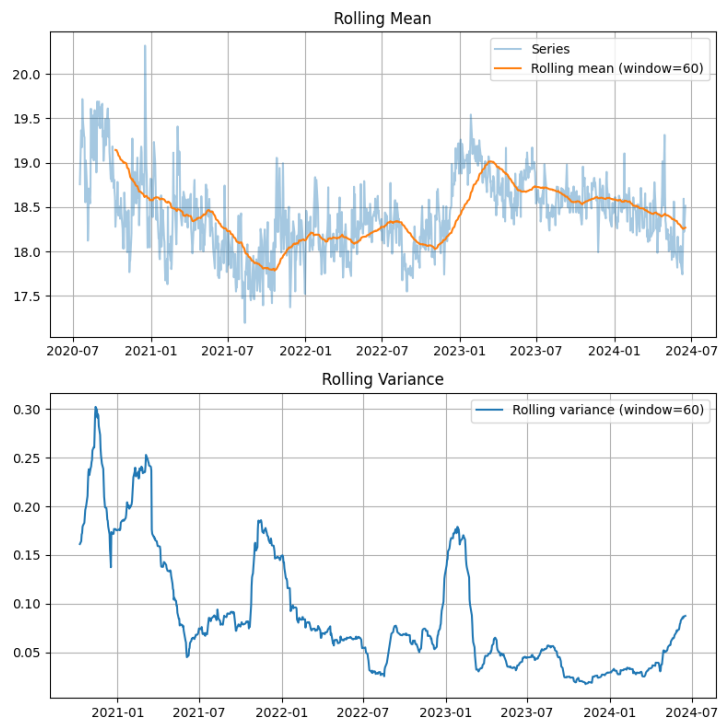


Figure 5: Rolling mean and variance (window 30) of $\log(\text{Volume})$.

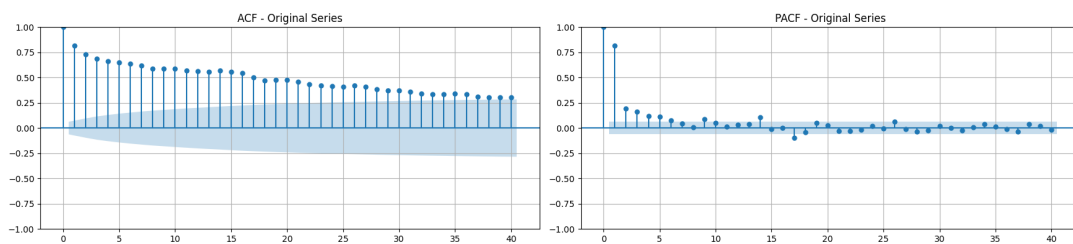


Figure 6: ACF and PACF of the log-volume series.

The rolling variance is highly unstable and the ACF of the log-series decays very slowly, suggesting that the log-level series may not be stationary, contrary to Jim’s claim.

Stationarity tests (ADF and KPSS)

We ran the ADF and both KPSS tests on the log-series y_t . The results were:

- **ADF**: statistic -3.1907 , p-value $0.0205 \Rightarrow$ reject H_0 of a unit root at 5%: *suggests stationarity*.
- **KPSS (level)**: statistic 0.5478 , p-value $0.0309 \Rightarrow$ reject H_0 of level stationarity: *suggests non-stationarity* at the 5% level.

This contradictory evidence, combined with unstable rolling variance and ACF behaviour, indicates that the log-volume series is *not reliably stationary*. An ARMA model on the raw or log-level series would therefore rest on shaky assumptions.

Stationarity after differencing

We consider the differenced series:

$$z_t = \Delta y_t = y_t - y_{t-1}.$$

The tests now yield fully consistent results:

- **ADF**: statistic -8.9551 , p-value $\approx 0 \Rightarrow$ very strong evidence of stationarity.
- **KPSS (level)**: p-value $\approx 0.10 \Rightarrow$ fail to reject stationarity.
- **KPSS (trend)**: p-value $\approx 0.10 \Rightarrow$ fail to reject stationarity.

Moreover, the ACF/PACF of $\Delta \log(\text{Volume})$ display rapid decay, with only a few significant lags, which is consistent with a low-order ARMA process.

The differenced log-series shows:

- full agreement between ADF and KPSS (clear stationarity),
- stabilised variance,
- short-memory ACF/PACF structure.

I examined seasonality using box plots and STL decomposition, and found no significant seasonal pattern. I therefore proceed with a non-seasonal model.

ARMA modelling of $\Delta \log(\text{Volume}_t)$

From the ACF/PACF of the transformed series, AR(5)–MA(2) appears plausible. To select the final model, we perform a grid search over $p, q \in \{0, 1, 2, 3, 4, 5\}$ and choose the specification with minimal AIC, applying an Occam's razor principle (prefer simpler models among near-equivalent candidates).

The lowest AIC is obtained for:

$$\Delta \log(V_t) \sim \text{ARMA}(1, 1).$$

	Coefficient	Std. Error	p-value
AR(1)	0.4209	0.035	< 0.001
MA(1)	-0.8542	0.022	< 0.001
σ^2	0.0572	0.002	< 0.001
AIC		-15.87	
BIC		-1.19	

Table 6: Estimated ARMA(1,1) model on $\Delta \log(\text{Volume}_t)$.

The model can be written as:

$$x_t = 0.4209 x_{t-1} - 0.8542 \varepsilon_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2).$$

Diagnostics of the ARMA(1,1) model

We now assess residual quality.

Normality. The QQ-plot and histogram exhibit heavy tails:

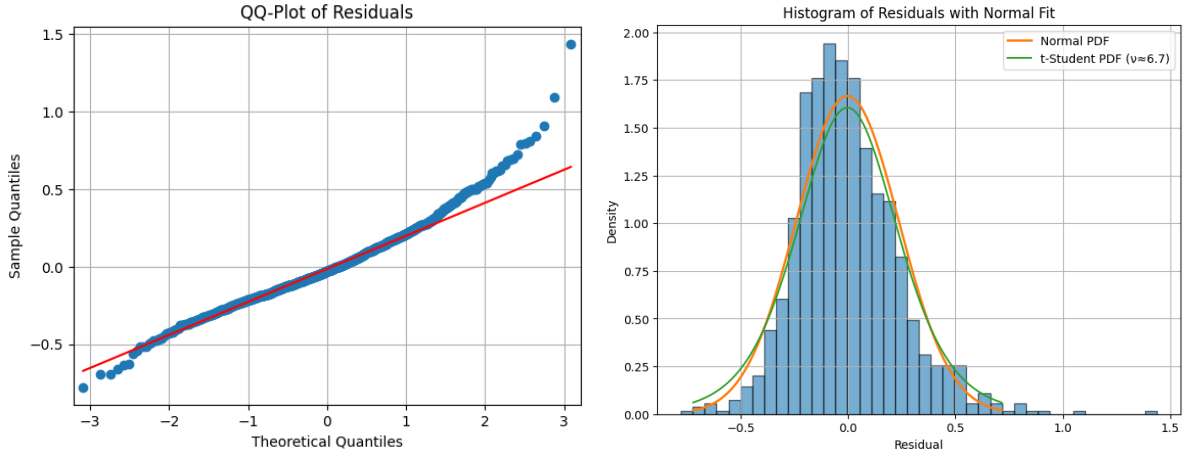


Figure 7: QQ-plot and histogram of ARMA residuals with normal and Student- t fits.

Residuals deviate significantly from normality, as both the Shapiro–Wilk test ($p < 0.001$) and the Jarque–Bera test ($p < 0.001$) strongly reject the hypothesis of normally distributed innovations.

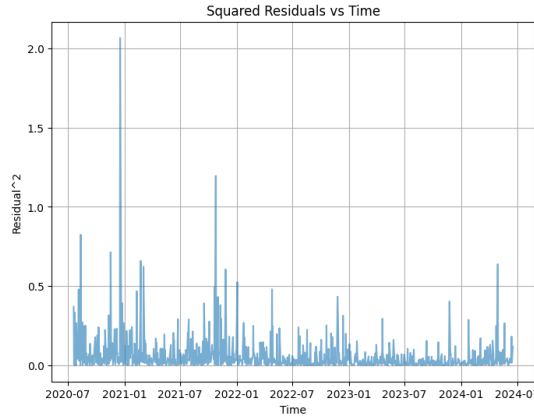


Figure 8: Squared residuals over time.

This plot highlights clear volatility clustering in the residuals, indicating that a GARCH-type model is required to capture the time-varying variance.

Autocorrelation. The Ljung–Box test on residuals shows no significant autocorrelation. The squared-residual tests clearly reject i.i.d. noise.

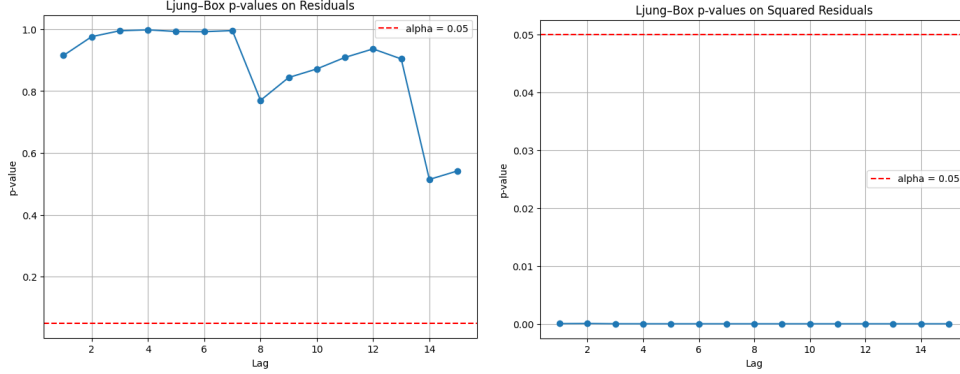


Figure 9: Ljung–Box p-values on squared residuals. Strong ARCH effects detected.

In addition, the ACF/PACF of squared residuals show significant lags. Therefore, the ARMA mean model is inadequate: volatility is clearly time-varying. This motivates replacing homoskedastic errors with a GARCH model.

GARCH modelling of log-differenced volumes

Because the ARMA residuals exhibit ARCH effects, we fit a GARCH model to the log-differenced series. Following standard practice, we approximate the MA component using a higher-order AR process in the mean equation. An AR(13) was selected based on information criteria. Candidate GARCH orders suggested by the residual squared ACF/PACF were (1,1), though we performed a grid search to confirm optimality.

We fit an AR(13)–GARCH(1,1) model with Student- t distributions:

$$\begin{aligned}
 x_t &= \mu + \sum_{i=1}^{13} \phi_i x_{t-i} + \varepsilon_t, \\
 \varepsilon_t &= \sigma_t z_t, \quad z_t \sim t_\nu, \\
 \sigma_t^2 &= \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2.
 \end{aligned}$$

Model performance.

- AIC: -141.74
- BIC: -44.20

Residual diagnostics. All diagnostics now support adequacy:

- No autocorrelation in residuals (Ljung–Box p-values > 0.1).
- No autocorrelation in squared residuals.
- ACF/PACF of residuals show no remaining structure.

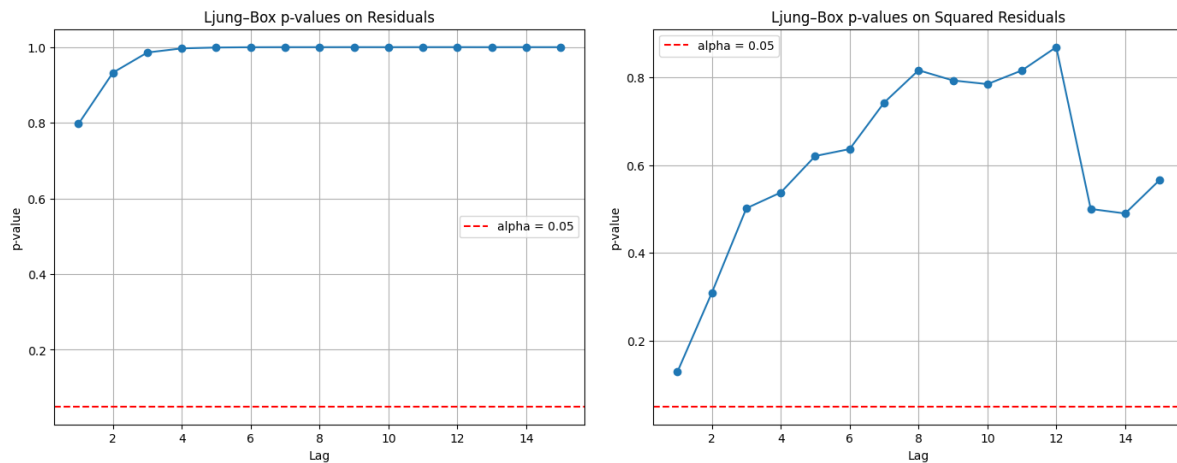


Figure 10: Ljung–Box on squared residuals for GARCH model.

Thus the GARCH model successfully removes the volatility clustering present in the ARMA residuals.

Therefore, the final recommended model for traded volumes is an AR–GARCH specification applied to log-differenced volumes. It provides a significantly better statistical description of the data than Jim’s ARMA model with homoskedastic innovations.

Note on implementation. For the time-series diagnostics, ARIMA modelling and GARCH residual analysis, I developed a small custom Python library that centralises all repetitive tasks: stationarity testing (ADF, KPSS, STL decomposition, rolling statistics), automatic ARIMA order selection (using AIC/BIC), and full residual diagnostics (QQ-plots, histogram fits, Ljung–Box tests on residuals and squared residuals, ACF/PACF of residuals, etc.). The library contains several long scripts, so it is not included in the report. Below, I show the calls used to generate the results.

```
# 1. Stationarity analysis (ADF, KPSS, STL, rolling stats)
res = check_stationarity(
    series = tradedvolumes["Volume"],
    rolling_window = 60,
    log_transform = True,
    difference = True,
    max_diff = 2,
    kpss_reg = "auto",
    stl_decompose = True,
    stl_period = 252,
)

# 2. Automatic ARIMA order selection on log-differenced series
res = arima_auto_select(
    series = tradedvolumes_diff,
    d = 0, # differencing already applied externally
    max_p = 5,
    max_q = 3,
    criterion = "aic", # or "bic"
)

p, _, q = res["best_order"]
trend = res["best_trend"]
print(f"Best ARIMA order: ({p}, 0, {q}), trend: {trend}")
```

```

best_model = res["best_model"]
best_model.summary()

# 3. Residual diagnostics of the selected ARIMA model
res = arima_residual_diagnostics(
    model      = best_model,
    alpha      = 0.05,
    show_plots = True,
    verbose    = True,
    acf_lags   = 30,
    pacf_lags  = 30,
    ljung_lags = (10, 20),
)

# 4. GARCH modelling on AR residual structure
arx = ARX(tradedvolumes_diff, lags = 15, rescale = False)
arx.volatility    = GARCH(1, 0, 1)          # candidate GARCH structure
arx.distribution = StudentsT()              # heavy-tailed residuals

garch = arx.fit(update_freq = 0, tol = 1e-5)
print(garch.summary())

```


6. Time-series analysis of ATM implied volatility ($T = 0.1$)

We now analyse the daily at-the-money implied volatility series with the objective of identifying an appropriate ARIMA-type model for its mean dynamics. The methodology follows the classical sequence: visual inspection, ACF/PACF, unit-root tests, differencing, ARIMA selection, and residual diagnostics.

ACF and PACF of the raw IV

The ACF of the raw series declines extremely slowly and remains positive across many lags, while the PACF displays a prominent lag-1 spike followed by gradual decay. Such behaviour is characteristic of a highly persistent or near-integrated process.



Figure 11: Rolling mean and variance (window 60) of Implied Vol.

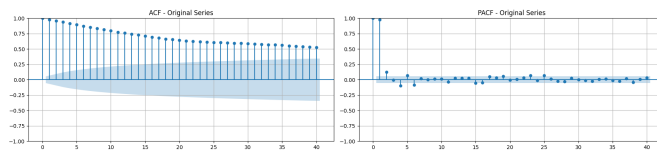


Figure 12: ACF and PACF of the ATM-IV series.

Both the unstable rolling variance and the slow decay of the ACF strongly suggest non-stationarity in the raw series.

Unit-root diagnostics (ADF and KPSS)

We jointly evaluate the ADF and KPSS tests:

- **ADF:** statistic = -3.87 , p-value = $0.002 \Rightarrow$ reject H_0 of a unit root (suggests stationarity).
- **KPSS:** statistic = 3.32 , p-value = $0.01 \Rightarrow$ reject H_0 of stationarity (suggests non-stationarity).

The contradiction between ADF (stationary) and KPSS (non-stationary), combined with unstable rolling moments and the slowly decaying ACF, indicates that the raw implied-volatility series is *not reliably stationary*. These symptoms are typical of borderline integrated or trend-stationary processes.

Stationarity after first differencing

We consider the first-differenced series:

$$z_t = \Delta\sigma_t = \sigma_t - \sigma_{t-1}.$$

The differenced data exhibit stabilised mean and variance, and the large COVID spike present in the raw series is removed.

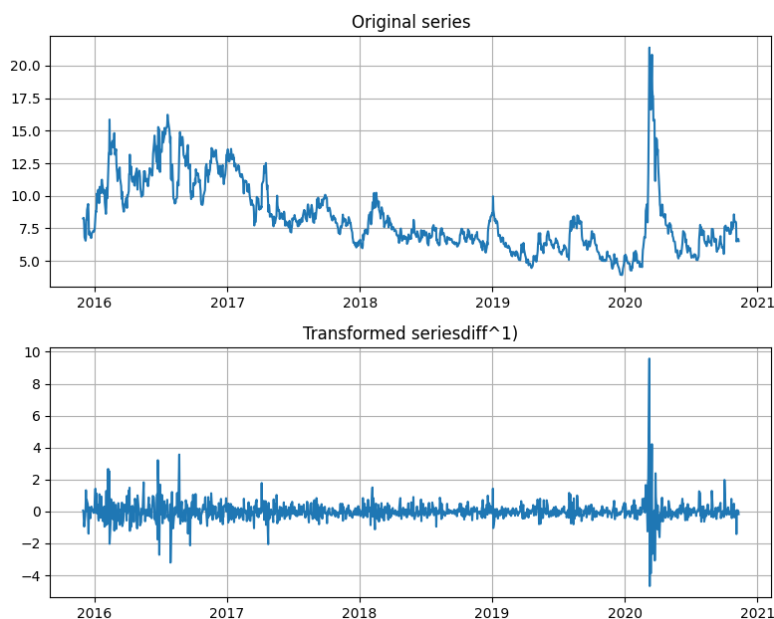


Figure 13: Original and differenced implied-volatility series.

Autocorrelations now decay rapidly, with only a few significant lags remaining. Unit-root tests now agree:

$$\begin{aligned} \text{ADF: stat} &= -9.84, p \approx 4.6 \times 10^{-17} && \Rightarrow \text{strong stationarity;} \\ \text{KPSS: stat} &= 0.017, p \geq 0.1 && \Rightarrow \text{fail to reject stationarity.} \end{aligned}$$

Hence, the appropriate differencing order is $d = 1$.

ARMA model selection on the differenced series

Since the differenced series z_t is stationary, we fit an ARMA model directly to z_t , which corresponds to an ARIMA($p, 0, q$) specification.

A grid search (or `auto_arima` with $d = 0$ on the differenced series) selects:

$\text{ARMA}(2, 2) \equiv \text{ARIMA}(2, 0, 2) \text{ on the differenced series}$

with $\text{AIC} = 2326.76$.

This choice is consistent with the short-memory structure visible in the ACF/PACF of $\Delta\sigma_t$.

Residual diagnostics

Autocorrelation. Residual ACF and PACF exhibit no remaining autocorrelation, indicating that ARMA(2,2) adequately captures the linear dependence in the differenced series.

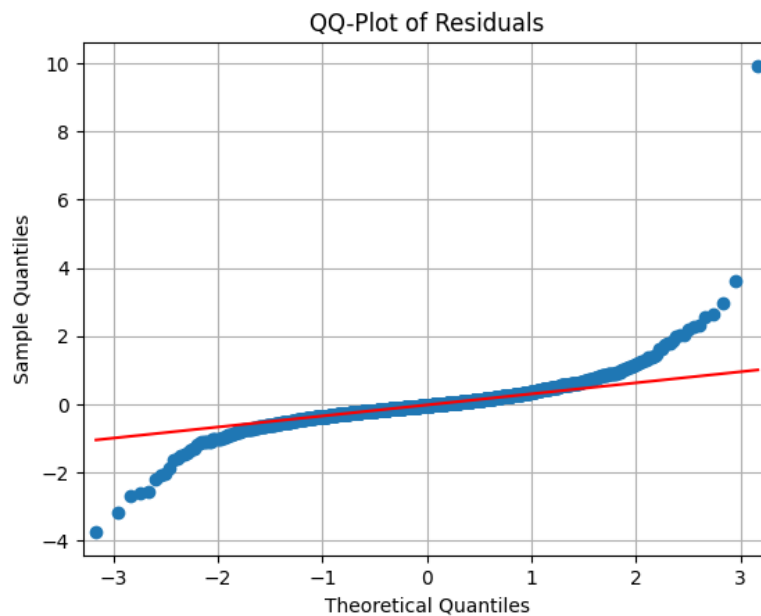


Figure 14: QQ plot of ARMA(2,2) residuals.

Ljung-Box test (residuals). All p-values exceed 0.05, so we do not reject the null of no remaining autocorrelation.

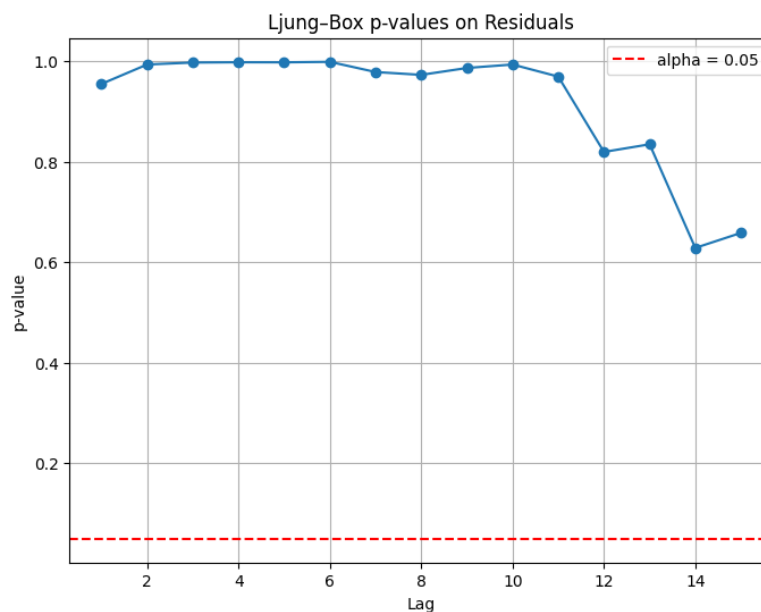


Figure 15: Ljung-Box p-values for ARMA residuals.

Ljung-Box test (squared residuals). Squared residuals exhibit extremely low p-values across many lags, indicating strong ARCH behaviour.

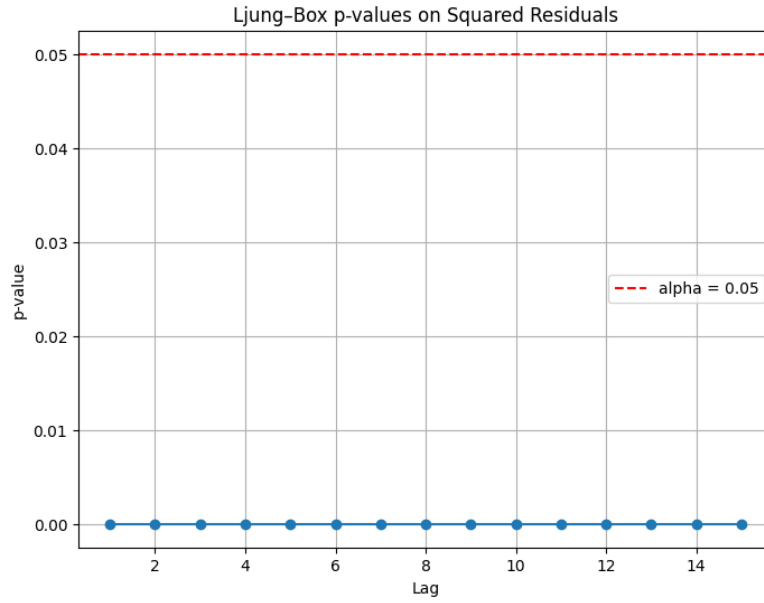


Figure 16: Ljung–Box p-values for squared residuals (strong ARCH effects).

- The raw ATM-IV series is not stationary.
- First differencing produces a stationary series with short-memory ACF/PACF.
- The best mean model on the differenced series is an ARMA(2, 2) model (i.e. ARIMA(2, 0, 2) on $\Delta\sigma_t$).
- However, strong ARCH signals in squared residuals show that volatility dynamics are not captured adequately.

Therefore, an ARMA–GARCH specification is more appropriate than a standalone ARIMA model. The ARMA(2,2) component models the mean of the differenced series, while a GARCH structure is required to capture the time-varying conditional variance.

2 Modelling Section

7. Discussion of drawbacks of the models considered in 2., 3. and 4., and possible improvements for interpolating between quoted option prices or implied volatilities

7.1 Data scarcity

All smile specifications at $T = 0.1$ are fitted on very small cross-sections (11 points for Jim's models, 46 for our TPB spline with $k = 6$ and $p = 10$). Even when a model fits well in-sample, the limited data makes parameters fragile and out-of-sample behaviour uncertain.

7.2 Residual diagnostics and correlated errors

A well-specified linear model should leave residuals that behave like i.i.d. noise. For the TPB cubic spline with $k = 6$, we examine linearity and homoscedasticity (residuals vs fitted), normality (QQ-plot, histogram), and independence (ACF, Ljung–Box).

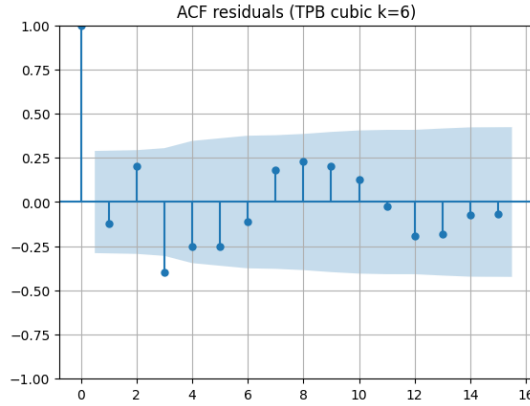


Figure 17: TPB cubic ($k = 6$): residuals vs fitted and evidence of autocorrelation.

Residuals are small and centred, consistent with the low RSS, but ACF and Ljung–Box tests indicate significant serial correlation.

Thus the OLS independence assumption is violated: if $\varepsilon \sim N(0, \Omega)$ with correlated errors, GLS

$$\hat{\beta}_{\text{GLS}} = (X^\top \Omega^{-1} X)^{-1} X^\top \Omega^{-1} y$$

provides more reliable standard errors, although specifying Ω correctly is difficult in practice.

7.3 Multicollinearity and rank assumptions

The TPB spline basis contains highly collinear functions $(1, x, x^2, x^3, (x - k_j)_+^3)$, especially over a narrow strike range.

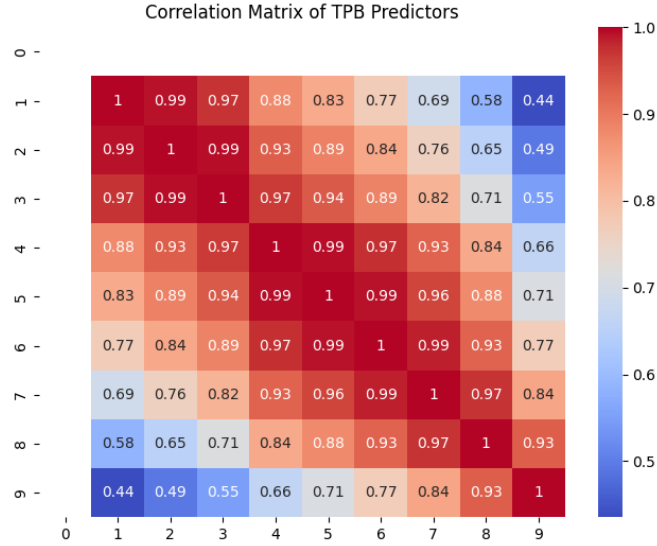


Figure 18: Correlation heatmap of TPB cubic spline predictors ($k = 6$).

Variance inflation factors (VIFs) confirm the issue:

Variable	VIF
0	5.20×10^6
1	9.70×10^6
2	1.01×10^8
3	7.35×10^7
4	4.04×10^6
5	3.18×10^5
6	5.38×10^4
7	8.10×10^3
8	7.67×10^2
9	3.19×10^1

Table 7: Variance Inflation Factors for TPB spline ($k = 6$).

Multicollinearity can make $X^\top X$ nearly singular:

$$\text{rank}(X) < p \implies X^\top X \text{ non-invertible.}$$

Consequences include unstable coefficients, inflated variances $\text{Var}(\hat{\beta}) = \sigma^2(X^\top X)^{-1}$, poor interpretability, and numerical ill-conditioning. In optimisation terms, if $\text{rank}(X) < p$, the loss $\|y - X\beta\|^2$ is convex but not strongly convex, producing an entire subspace of minimisers.

7.4 Overfitting

With $p = 10$ parameters for only 46 points, the TPB spline with $k = 6$ may overfit. High flexibility combined with multicollinearity reinforces this risk. We therefore compare training error and 5-fold CV error on `newquotes_`, and assess out-of-sample performance on the 11 points from `impliedvols.csv`. A persistent gap between train and validation MSE would indicate overfitting.

7.5 Convexity, smoothness and the modelling domain (IV vs call price)

An arbitrage-free call price surface must be non-increasing and convex in strike ($C''(K) \geq 0$). The piecewise-linear IV model breaks smoothness and may violate convexity; Jim's one-knot

cubic has $C''(K)$ of mixed sign. Our TPB cubic spline, fitted on IV and mapped to call prices, yields a numerically non-negative finite-difference $C''(K)$ across strikes:

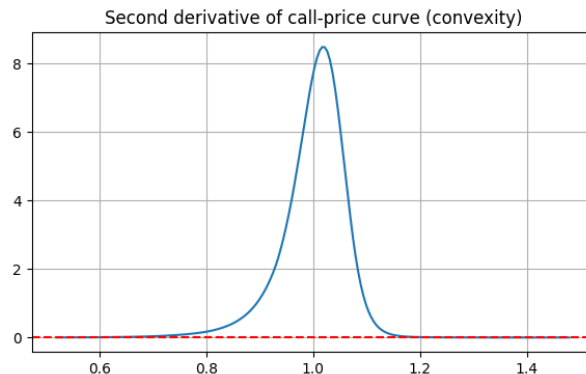


Figure 19: Finite-difference approximation of $C''(K)$ for TPB cubic spline ($k = 6$).

Possible improvements and their limitations

Several methods can mitigate these issues, each with trade-offs:

(i) Enforcing convexity in the call-price domain. Requiring $C''(K) \geq 0$ ensures no-arbitrage but turns the problem into a constrained optimisation. The solution is typically less smooth and computationally heavier than OLS.

(ii) Using GLS for correlated residuals. GLS corrects for serial correlation and yields valid inference, but specifying Ω (AR(1), exponential decay, etc.) is difficult in a cross-section and may introduce new biases. GLS is also less stable under multicollinearity.

(iii) Ridge or Lasso regularisation. Ridge stabilises estimates when $X^\top X$ is ill-conditioned, reducing variance at the cost of bias. Lasso can remove redundant knots but may oversimplify curvature, producing non-smooth smiles.

(iv) Richer non-linear models. Gaussian kernels provide smooth fits and capture complex curvature, but do not guarantee convexity and may behave poorly with sparse strikes. More advanced ML models (boosting, bagging, neural nets) increase flexibility but reduce interpretability and require more data, making them less suitable for transparent smile interpolation.

8.i) Propose and implement a model to be used instead of the available solutions and show, using appropriate criteria, why yours is better

The cubic spline model fitted on `newquotes.csv` and tested on `impliedvols.csv` already delivers an almost perfect out-of-sample fit. Both datasets are highly correlated, so training on `newquotes` produces very small prediction errors. Observed-vs-Predicted points lie almost exactly on the diagonal, and residuals are tightly centred around zero, confirming that the spline captures the smile extremely well.

Starting from this strong baseline, I tested several extensions to evaluate whether the model could be improved.

Ridge and Lasso regularisation. Refitting the spline with Ridge and Lasso penalties and tuning λ via grid search shows that the optimal λ is essentially zero. The coefficient paths remain almost flat (Figure 20), showing that regularisation brings no advantage: the baseline spline already has low variance, and penalisation would only add unnecessary bias.

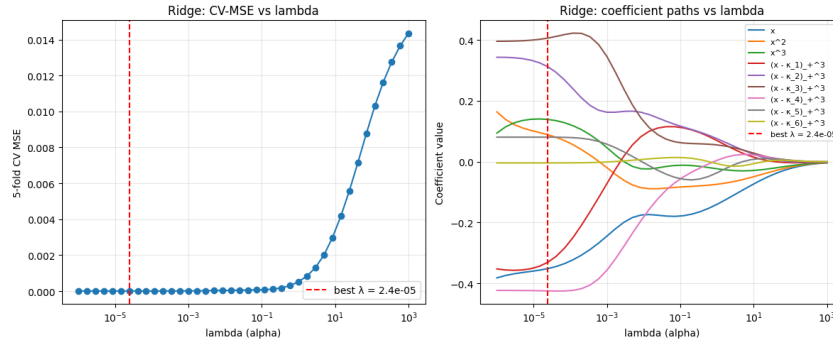


Figure 20: Ridge coefficient paths $\beta_j(\lambda)$ for the cubic spline. The optimal λ is nearly zero, so the solution is effectively OLS.

GLS for autocorrelated residuals. Because a Ljung–Box test indicated mild autocorrelation, a GLS version with an AR(1) error structure was tested. However, the GLS fit is numerically unstable and does not improve the test MSE or AIC.

PCA on an enriched spline basis. I also considered a larger spline basis (more knots), applied PCA, and retained the components explaining 90% of variance before refitting the regression. This produces similar test performance but with higher complexity and reduced interpretability, so it is not retained.

Table 8 summarises the results. The plain cubic spline achieves the best or tied-best test MSE and AIC. Given its simplicity, stability and excellent out-of-sample behaviour, it remains the preferred model.

Table 8: Comparison of cubic spline variants at $T = 0.1$ (train and test).

Model	Train MSE	Test MSE	Train AIC	Test AIC
Cubic spline (OLS)	0.000003	0.000010	−567.19	−127.72
Cubic + GLS	0.000003	0.000030	−565.42	−119.87
Cubic + Ridge	0.000003	0.000200	−566.69	−125.98
Cubic + Lasso	0.000005	0.000037	−534.98	−102.59
Cubic + PCA basis	0.093466	0.109270	−107.03	−22.35

8.ii. Potential drawbacks of the chosen model in production

The main limitation of the cubic spline model is its sensitivity to the choice and placement of knots. If the strike grid changes or becomes sparse, the spline may oscillate or behave poorly in the wings, making the smile unstable under small perturbations.

Another drawback is that spline coefficients are not directly interpretable and must be refitted each day. As a result, the volatility surface does not evolve smoothly over time, which can be problematic in production systems where temporal consistency is required.

Regularisation methods (Ridge/Lasso) also have subtle drawbacks in this setting. Penalisation reduces variance at the cost of added bias, but here the training and test datasets are extremely similar and highly correlated. Because variance is already very low, regularisation only increases

bias without improving stability—hence the optimal tuning parameter collapses to (almost) zero.

(9.i) Time-series model for the ATMTsIV.csv series at maturity $T = 0.25$

Preprocessing and graphical inspection

The daily ATM IV series ($T = 0.25$) is strictly positive, highly persistent, and displays slow trends and sharp volatility bursts (notably in 2020). Rolling mean and variance drift substantially. The ACF decays extremely slowly (above 0.7 at lag 40) and the PACF has a dominant lag-1 spike (Fig. 21), all of which is inconsistent with stationarity.

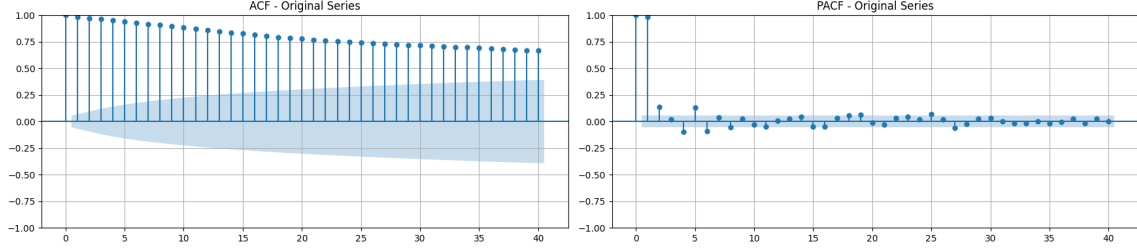


Figure 21: ACF (left) and PACF (right) of the level ATM IV series.

Stationarity tests

ADF fails to reject a unit root, while KPSS rejects stationarity: the level series is clearly non-stationary.

Differencing

We consider $z_t = \Delta y_t$. After differencing, ADF rejects a unit root and both KPSS tests fail to reject stationarity. Thus z_t behaves as a stationary short-memory process.

I examined seasonality using box plots and STL decomposition, and found no significant seasonal pattern. I therefore proceed with a non-seasonal model.

Selected ARMA model

The ACF and PACF of Δy_t show only a few significant lags, suggesting a low-order ARMA. A grid search over $p, q \in \{0, 1, 2, 3, 4\}$ selects:

$$z_t \sim \text{ARMA}(2, 2).$$

$$z_t = \phi_1 z_{t-1} + \phi_2 z_{t-2} + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \varepsilon_t.$$

	Coefficient	Std. Error	p-value
AR(1)	-1.4032	0.025	< 0.001
AR(2)	-0.6773	0.028	< 0.001
MA(1)	1.2897	0.028	< 0.001
MA(2)	0.4990	0.031	< 0.001
σ^2	0.1389	0.001	< 0.001
AIC		1125.36	
BIC		1151.18	

Table 9: Estimated ARIMA(2,0,2) model for $T = 0.25$ ATM IV.

The AR terms imply strong short-term mean reversion, while MA terms absorb short-lived shocks.

Diagnostics

Residuals are heavy-tailed: histogram and QQ-plot favour a Student- t shape, and Jarque–Bera rejects normality (Fig. 22). Ljung–Box tests show no autocorrelation in *raw* residuals, but squared residuals exhibit strong dependence (Fig. 23), revealing clear ARCH effects.

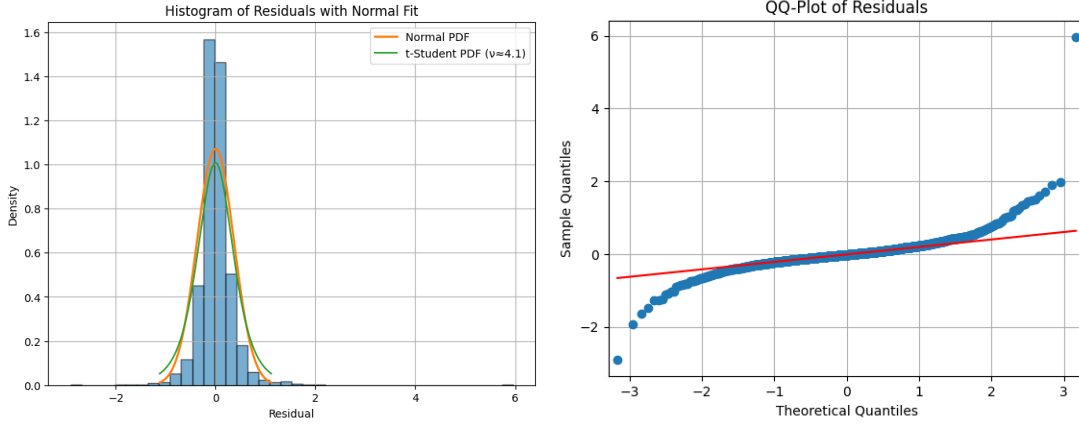


Figure 22: Histogram and QQ-plot of ARIMA residuals.

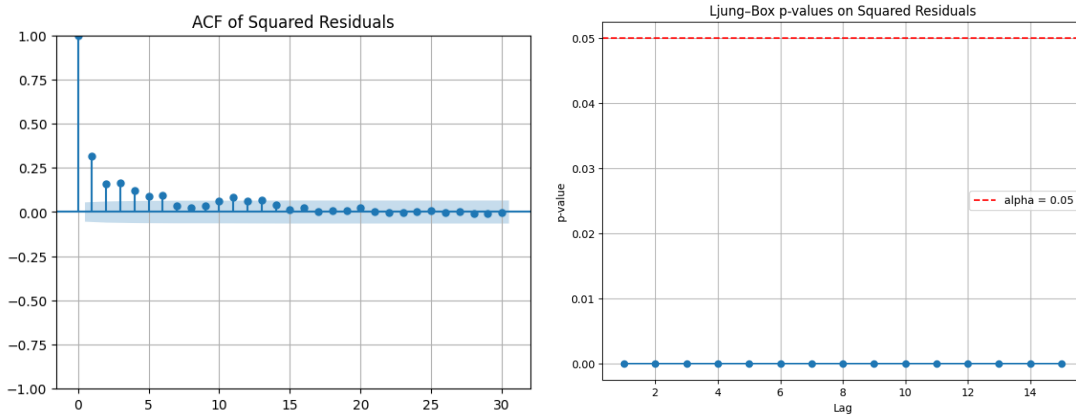


Figure 23: Squared-residual ACF and Ljung–Box p-values: strong ARCH effects.

The mean is well specified, but the variance is clearly time-varying.

GARCH modelling of ATM implied volatilities

Given the strong ARCH effects and heavy tails, we extend the model with a conditional variance process. To approximate the MA(2) term, an AR(15) mean is used. A grid search selects GARCH(1, 0, 2), and Skew-Student innovations provide the best fit.

$$\begin{aligned}
 y_t &= \mu + \sum_{i=1}^{15} \phi_i y_{t-i} + \varepsilon_t, \\
 \varepsilon_t &= \sigma_t z_t, \quad z_t \sim \text{Skew-Student}(\nu, \xi), \\
 \sigma_t^2 &= \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2 + \beta_2 \sigma_{t-2}^2.
 \end{aligned}$$

Model performance. The AR–GARCH model is strongly favoured by information criteria:

$$\text{AIC} = 19.2562, \quad \text{BIC} = 132.589.$$

Residual diagnostics. Residuals show:

- no remaining autocorrelation (Ljung–Box p-values > 0.1);
- no ARCH effects (p-values on squared residuals > 0.05);
- heavy tails well captured by Skew–Student innovations.

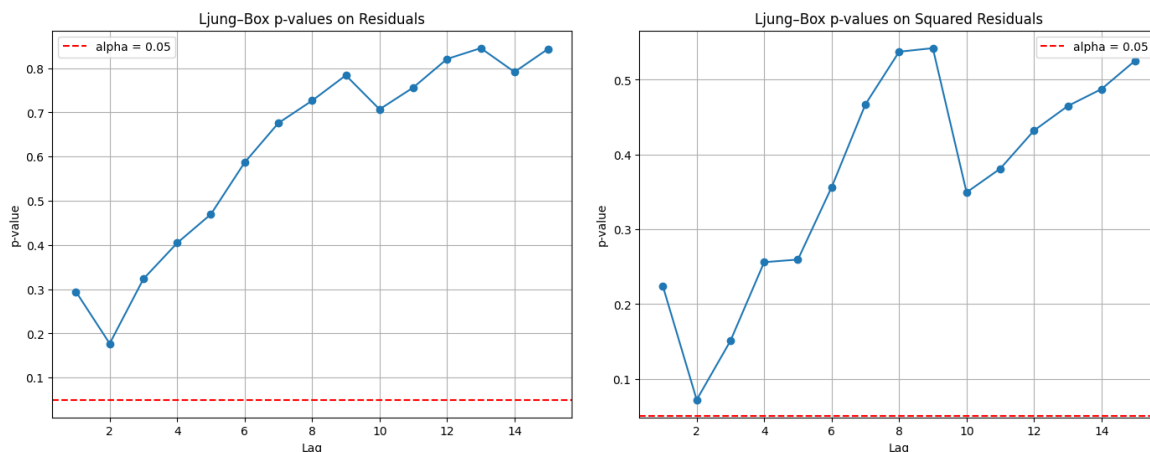


Figure 24: Ljung–Box p-values on residuals (left) and squared residuals (right).

The GARCH component removes volatility clustering, and Skew–Student errors capture the heavy-tailed nature of IV innovations. **The recommended model for ATM IV at $T = 0.25$ is therefore the AR(15)–GARCH(1,0,2) with Skew–Student innovations.**

9.ii) Dimension reduction of ATM term–structure movements (PCA)

To simplify the joint modelling of ATM implied volatilities across maturities, we apply PCA to the *daily differences* of the ATM–IV curve. Working on differences removes level effects and isolates the actual movements of the curve.

The variance ratio plot shows that the first component explains almost all of the variation in daily changes (around 90%). This confirms that the seven maturities do not move independently, but follow a small number of coherent patterns.

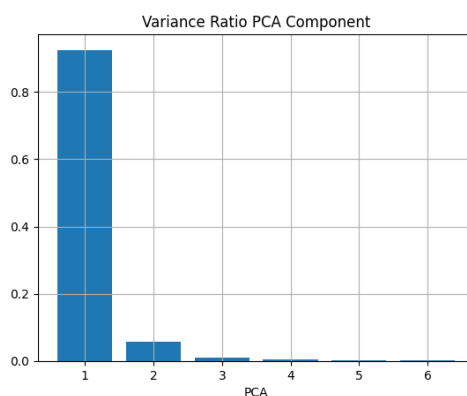


Figure 25: Explained variance ratio of PCA components.

PC1: Parallel shift. The loadings of PC1 are strictly positive and decreasing with maturity, indicating that the dominant movement of the term-structure is a *parallel shift*: all maturities move up or down together, with slightly larger impact on the short end.

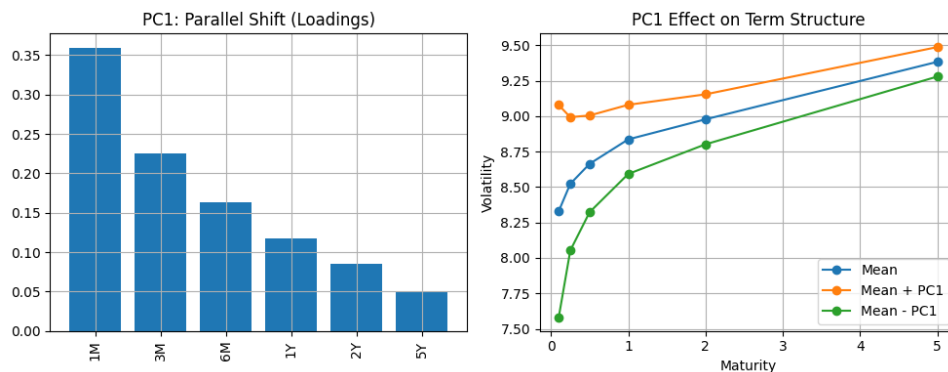


Figure 26: PC1 loadings and its effect on the term structure.

PC2: Slope / rotation. PC2 changes sign between short and long maturities, capturing a *tilting* of the curve. This corresponds to steepening or flattening moves (short maturities increasing while long maturities decrease, or vice versa).

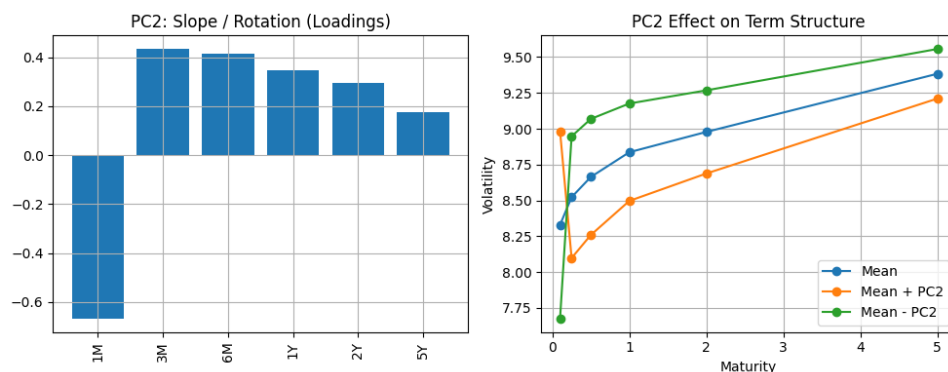


Figure 27: PC2 loadings and resulting slope/rotation effect.

PC3: Curvature / hump. PC3 loadings exhibit opposite signs on the mid part versus the wings. This represents a *curvature* adjustment, where intermediate maturities move differently from the short and long ends.

Together, these three components provide a compact and interpretable representation of the joint dynamics of ATM implied volatilities: (1) a global level move, (2) a slope adjustment, and (3) a curvature effect. Instead of modelling seven correlated time series, we can model only the three uncorrelated PCA factors, which captures almost the entire structure of daily variation. This offers a highly efficient simplification of the time-series model for multi-maturity implied volatilities.

Because each instrument has a DV01 (or vega) at its maturity, its exposure to a given component is simply its sensitivity multiplied by the PC loading at that maturity. Summing these exposures gives the portfolio's net exposure to PC1, PC2 and PC3. By taking offsetting positions in a small set of liquid maturities (e.g. 0.5Y, 1Y, 5Y), one can choose notionals that neutralise these exposures—producing a portfolio that is hedged against level moves (PC1), slope changes (PC2) or curvature shocks (PC3).